

Course: Speech Recognition and Generation course

Assignment: Automatic Speech Recognition — Decoding Strategies

Author: Natalya Muravyova

University: ITMO, AITH

Date: April 2025

1. Introduction

In this assignment, I implement and compare four decoding strategies for automatic speech recognition (ASR) using a pre-trained wav2vec2 model trained on the LibriSpeech dataset. The goal is to evaluate decoding quality using character-level Levenshtein distance and to observe how different decoding methods and language model (LM) configurations influence recognition performance.

The decoding strategies explored in this report are:

- Greedy decoding
- Beam search decoding
- Beam search with language model (LM) fusion
- Beam search with second-pass LM rescoring

Additionally, I evaluate how model quality changes with:

- Different beam search parameters (`beam_width`, `alpha`, `beta`)
- Different sizes of N-gram KenLM models (3-gram vs 4-gram)

Test data consists of arbitrary 16 kHz English audio files paired with ground-truth uppercase transcriptions.

2. Setup and Tools

- **Model:** `facebook/wav2vec2-base-960h`
- **LMs used:** 3-gram (pruned), 4-gram (unpruned)
- **Tokenizer:** Based on CTC vocabulary with space as the word boundary (`|`)
- **Libraries:** `torch`, `torchaudio`, `transformers`, `kenlm`, `Levenshtein`, `heapq`

Again, test audio files were sampled at 16 kHz, and decoded text was compared to ground truth transcripts using normalized Levenshtein distance.

3. Implementation

3.1 Greedy and Beam Decoding

The greedy and beam decoders were implemented by selecting the most probable token (or top-K hypotheses) from the logit matrix output by the acoustic model.

3.2 Beam with LM Fusion

This method adds log-probabilities from a language model (KenLM) during beam expansion, controlled by the `alpha` parameter.

3.3 Beam with LM Rescoring

A second-pass rescoring of the beam hypotheses using the LM was implemented. This allows for more accurate hypothesis selection after decoding.

4. Language Model Comparison

To assess the impact of language model size, both the default 3-gram and a larger 4-gram model were used. All experiments in this section were conducted with fixed decoding parameters:

- `beam_width = 3`
- `alpha = 1.0`
- `beta = 1.0`

The evaluation was performed by comparing decoded hypotheses to ground truth using character-level Levenshtein distance.

Observations

- **Greedy** and **beam search** results remained identical across both 3-gram and 4-gram configurations. This behavior was expected, as these decoding strategies do not incorporate a language model.
- **beam_lm**:
 - In the 4-gram setup, performance did not improve.
 - In some cases, it degraded compared to the 3-gram model.
 - On average, character error rates either stayed the same or slightly increased.
- **beam_lm_rescore**:
 - The results with 4-gram were almost always identical to those with the 3-gram model.
 - No consistent gain was observed, but this method remained stable.

Conclusions

- The 4-gram language model **does not provide meaningful improvements** over the 3-gram LM in this setting.
- In fact, it may **degrade beam_lm performance**, possibly due to overfitting or mismatch between the LM and the beam hypotheses.
- The **beam_lm_rescore** method is more stable, but offers **no consistent benefit** when switching to the 4-gram LM.

5. Parameter Tuning

To better understand the behavior of decoding algorithms, we conducted a systematic study by varying the key decoding parameters:

- **Beam width** (**beam_width**): [3, 5, 10]
- **Language model weight** (**alpha**): [0.5, 1.0, 1.5]
- **Length penalty** (**beta**): [0.0, 0.5, 1.0]

For each combination, we measured the decoding quality using character-level Levenshtein distance across four decoding strategies:

- **greedy**
- **beam**
- **beam_lm**
- **beam_lm_rescore**

General Observations

- **greedy**, **beam**, and **beam_lm_rescore** decoding methods behaved identically across both language models (3-gram and 4-gram).
- Their results remained **stable with a consistent Levenshtein distance of ~10.0**, independent of the chosen parameters.

Key Differences in **beam_lm** Behavior

The **beam_lm** method was the only one where parameter tuning and language model size had a **notable impact**.

Improvements with the 4-gram model:

- At **alpha = 0.5**, **beta = 1.0**, the 4-gram model achieved the **lowest distance of 9.625**, compared to **9.75** for the 3-gram.
- At **alpha = 1.0**, **beta = 1.0**, the 4-gram also performed **slightly better: 10.75 vs 10.875** for the 3-gram.

Degradation patterns:

- With **alpha = 1.5**, performance with the 4-gram model started to decline — **as expected due to overweighting the LM**.
- However, the 4-gram model still showed **less degradation on average**:
 - Maximum distance: **12.875** for 4-gram vs **13.625** for 3-gram.
 - The 4-gram model was **more resilient** at higher **alpha** values, especially with larger **beam_width**.

Conclusion

- **The 4-gram LM provides moderate benefits**:
 - Slightly better quality at lower **alpha** values.
 - Slightly more robust behavior at high **alpha**, particularly with large **beam_width**.

- However, these improvements are **not dramatic**. The effect of switching from a 3-gram to a 4-gram model is only noticeable in `beam_lm` and **heavily depends on parameter tuning**.
- Therefore, careful tuning of `alpha` and `beta` remains **critical** when using language model fusion.
- Using a 4-gram model may be **justified in scenarios where `beam_lm` is applied and character-level accuracy is important**, but **not essential** otherwise.

6. Conclusion

In this assignment, I explored and implemented several decoding strategies for the wav2vec2 acoustic model, with and without the use of external language models. The key findings are as follows:

- **Greedy, beam, and beam with LM rescoring** decoding methods showed **consistent and stable performance** across all tested configurations and were **largely unaffected by LM size or decoding parameters**.
- The **beam with shallow LM fusion** method (`beam_lm`) demonstrated the most sensitivity to parameter tuning and language model characteristics:
 - It showed **modest improvements** when using a 4-gram LM with low alpha values (e.g., `alpha=0.5`, `beta=1.0`).
 - However, it also showed **performance degradation** at higher alpha values, especially with the 3-gram model.
- The 4-gram language model did not bring significant improvements over the 3-gram model in most cases. Its benefits were **marginal** and limited to specific parameter configurations.
- Parameter tuning — particularly the selection of `alpha` and `beta` — is **crucial** when applying LM fusion. Without tuning, LM fusion may even harm performance.

Overall, while language model integration can enhance decoding quality in some scenarios, **its impact is modest** in this setup. For most robust and consistent results, **beam decoding or LM rescoring with conservative parameters** is a safer choice.